



CTD Python Essentials

Week 12: More Python Skills



Comprehensions

- Comprehensions are often the most concise and clear way to specify iteration in python
- They can be used to create
 - lists # [*value* ...]
 - dicts # { *key* : *value* ...}
 - Sets # {*value* ...}
- Can be nested
- They can include filtering

```
comprehension ::= assignment_expression comp_for
comp_for      ::= ["async"] "for" target_list "in" or_test [comp_iter]
comp_iter     ::= comp_for | comp_if
comp_if       ::= "if" or_test [comp_iter]
```

List Comprehension Examples

- Code in this [Github Gist](#)

```
squares = [n**2 for n in range(10)]  
# => [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
matrix = [[1, 2, 3], [4, 5, 6]]  
evens = [x                                # result expression  
         for row in matrix                # outer loop  
         for x in row                    # inner loop  
         if x % 2 == 0]                  # filter  
# [2, 4, 6]
```

```
colors = ["red", "green", "blue"]  
sizes = ["S", "M", "L"]  
inventory = [f"{size}-{color.upper()}"  
             for color in colors  
             for size in sizes  
             if not (color == "green" and size == "L")]  
# ['S-RED', 'M-RED', 'L-RED', 'S-GREEN', 'M-GREEN',  
#  'S-BLUE', 'M-BLUE', 'L-BLUE']
```

```
# nested list comprehension  
rows, cols = 3, 4  
mult_table = [[(r+1)*(c+1)  
               for c in range(cols)]  
              for r in range(rows)]  
# [[1, 2, 3, 4],  
#  [2, 4, 6, 8],  
#  [3, 6, 9, 12]]
```

Dictionary Comprehensions Examples

- Code in the same [Github Gist](#)

```
# Simple dictionary comprehension
cubes = {n: n**3 for n in range(6)}
# {0: 0, 1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
```

```
# word length for unique words >= 4 chars
words = ["chat", "gpt", "python", "chat", "code"]
length_by_word = {w: len(w) for w in words if len(w) >= 4}
# {'python': 6, 'code': 4, 'chat': 4}
```

Set Comprehensions

- Code in the same [Github Gist](#)
- Sets and set comprehensions are often used to filter duplicates

```
emails = [  
    "alice@example.com",  
    "Bob@Example.com",  
    "carol@sub.mail.co.uk",  
    "dave@example.com",  
    "eve@test.org",  
    "mallory@Example.com"  
]  
  
# unique domains  
domains = {addr.split("@")[1].lower() for addr in emails}  
# {'sub.mail.co.uk', 'example.com', 'test.org'}
```


Generators

- A [generator](#) is any Python object that implements the *iterator protocol* but produces its values lazily
 - Items are computed only when the caller asks for the next one
- Can be created by
 - A [generator function](#) containing yield
 - A [generator expression](#)
- Can be used to create efficient streaming pipelines
 - Large datasets and files
 - Don't need to keep the whole list in memory

Generator examples

- Code in the same [Github Gist](#)

```
# simple generator expression
# a million numbers but only 10 created
evens = (x for x in range(1_000_000) if x % 2 == 0)
first_ten = [next(evens) for _ in range(10)]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
print(first_ten)
```

```
# simple generator function
def squares(n):
    for i in range(n):
        yield i * i

for s in squares(5):
    print(s)                # 0 1 4 9 16
```

```
## looking at part of an infinite stream
import itertools
# Fibonacci series
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
print(list(itertools.islice(fib(), 20)))
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144,
# 233, 377, 610, 987, 1597, 2584, 4181]
```

Decorators

- Decorators provide a concise way to create wrapper functions
 - Function composition
 - Code reuse
 - Can also replace the target (as in [@property](#))
- Run only once at definition time
- There are many built-in decorators
- Example code [here](#)

Decorator examples

```
import time
from functools import wraps

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.3f}s")
        return result
    return wrapper

import random

@timed
# a poor sorting algorithm which is slow
def bubble_sort(seq):
    seq = list(seq)          # work on a copy
    n = len(seq)
    for i in range(n):
        for j in range(n - i - 1):
            if seq[j] > seq[j + 1]:
                seq[j], seq[j + 1] = seq[j + 1], seq[j]
    return seq

nums = [random.random() for _ in range(5_000)]
bubble_sort(nums)
```

```
# Access methods as properties
class Rectangle:
    def __init__(self, w, h):
        self._w, self._h = w, h

    @property
    def area(self):
        return self._w * self._h

r = Rectangle(10, 5)
print(r.area)
```

```
# Handy builtin decorators for classes

# Automatically generate dunder methods
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(3, 4)
print(p)          # Point(x=3, y=4) (__repr__ auto-generated)
print(p.x + p.y)  # dataclass added __init__
```

Closures

- Like many other languages, python provides closures
- When a function is created, it has access to the environment which is in scope when it was created.
- Reminder that python scoping is at the function level
- Functions defined inside other functions can access the variables from the outer function(s)

```
def make_power(exponent):  
    def power(base):  
        return base ** exponent  
    return power  
  
square = make_power(2)  
cube   = make_power(3)  
  
print(square(5))    # 25  
print(cube(5))      # 125
```

Classes and Objects

- Everything in python is an object
- Every object is an instance of a particular class
- Python provides lots of support for creating your own classes and doing object oriented programming
 - Datatypes match the problem
 - Abstraction
 - Match real-world concepts
 - Encapsulation helps organize code
 - Natural modularization
- Methods are functions provided by a class
 - Operate on the data with instances of the class
- Python classes are configured using a wide variety of “dunder” methods
 - From “double underscore”
 - E.g. `__init__(self)` is used to create and initialize an object

Classes and Objects

```
# Point class example
import math
class Point:
    """A 2-D class."""

    def __init__(self, x: float, y: float):
        self.x = float(x)
        self.y = float(y)

    # print representation
    def __repr__(self) -> str:
        return f"Point(x={self.x}, y={self.y})"

    # string conversion
    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    # Allows unpacking
    def __iter__(self):
        yield from (self.x, self.y)

    # comparison operator
    def __eq__(self, other):
        if isinstance(other, Point):
            return (self.x, self.y) == (other.x, other.y)
        return NotImplemented
```

```
# Make hashable
__hash__ = object.__hash__

# Distance computation
def distance_to(self, other: "Point") -> float:
    """Euclidean distance."""
    return math.hypot(self.x - other.x, self.y - other.y)

# minimum routing distance on a chip
def manhattan_distance(self, other: "Point") -> float:
    """Taxicab distance"""
    return abs(self.x - other.x) + abs(self.y - other.y)

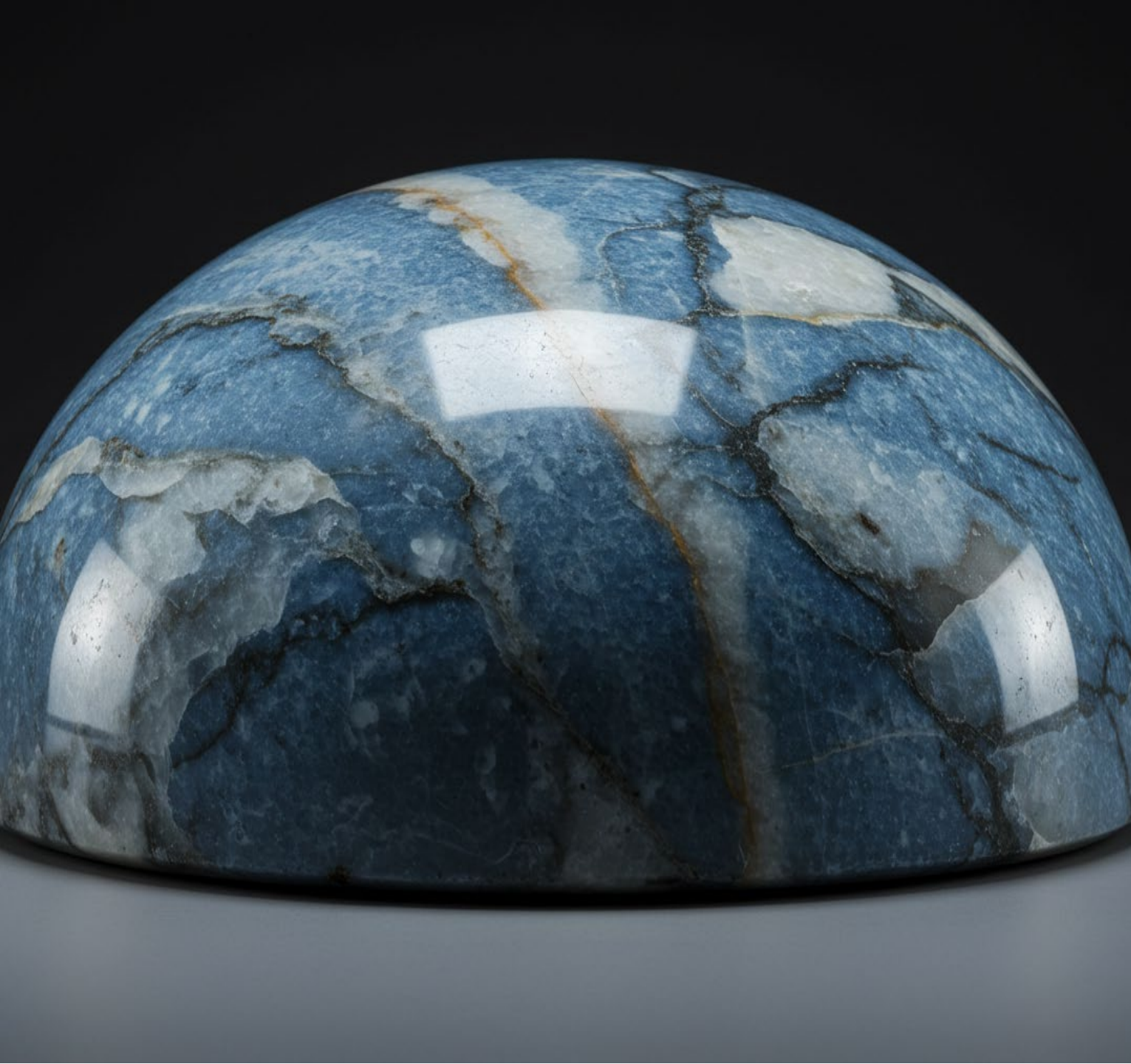
p = Point(3, 4)
print(p)

q = Point(6, 8)

euclidian = p.distance_to(q)          # Euclidean
# 5.0

manhattan = p.manhattan_distance(q)   # Manhattan
# 7.0

print(euclidian, manhattan)
```



Capstone Project

- The final three weeks will provide an opportunity to use skills from the class to create capstone projects
- Two projects
 - Kaggle base data pipeline
 - Student choose one of four dataset and perform cleaning, aggregation, analysis and presentation
 - Alternatively use another dataset with CIL approval
 - Local project published on github
 - Scraping, persistence, analysis and publication in a dashboard
 - Use either Dash or Streamlit

Q&A and Demo

